Climate Hazard Trend Analyzer

Implementation report · Full-stack case study submission

Stack	Python 3.11, Django 5.1, Django REST Framework, MySQL (MariaDB 11.8 tested), React 18, Vite, Chart.js, Leaflet
Data source	Open-Meteo Historical Weather API (ERA5 reanalysis), Open-Meteo geocoding; no API key required
Repository	Public GitHub link: to be added at submission
Deployed app	To be added at submission (not deployed yet)
Status	Feature-complete for the case scope; backend tests passing; UI verified in a browser for the main flows

1. Summary

The application analyses how climate hazards have changed for a chosen location over a chosen period. A user picks a place (by city search, by clicking a map, or by entering coordinates), a hazard, and a year range. The backend fetches historical daily weather, caches it in MySQL, detects events with a documented definition, computes yearly counts and trend statistics, and returns plain-language insights and chart-ready data. The React frontend presents the summary, charts, data-quality information, and exports.

Three hazards are implemented: **heatwave** (default), **heavy rainfall**, and **drought** (dry spells). Regions can be a single point or a box, where a box is analysed as the mean of a 3×3 grid of points.

2. Architecture

```
React (Vite)  --/api-->  Django REST API  -->  Open-Meteo archive   (only days not yet cached)
  RegionPicker (city search, map, coordinates)     |
  Form + KPIs + insights                           +-- ingest.py    clean, bulk-upsert daily rows  -->
MySQL
  Chart.js charts, CSV / print                     +-- detection.py  generic run-length event detector
                                                   +-- heatwave.py  heatwave / heavy-rain / drought
rules
                                                   +-- trends.py    OLS, Mann-Kendall, Sen's slope
                                                   +-- analysis.py  orchestration, result document -->
MySQL
```

The backend is organised as one Django app (`hazards`) inside a Django project (`climate_api`). Each module has one responsibility, so the hazard rules can be tested without the network or the database.

3. Data sources and ingestion

- **Daily variables:** maximum and minimum temperature at 2 m (°C) and precipitation sum (mm), timezone GMT.
- **Why Open-Meteo:** no key, a long record (1940 to a few days ago), and one request covers a whole date range per location.
- **Caching in three layers:**

1. `DailyWeather`: one row per location and day, unique on (location, date). Repeat requests read from here.
2. `FetchedRange`: records which date ranges were already downloaded, so a covered range is never re-fetched.
3. `AnalysisResult`: the full result JSON, keyed by a SHA-256 hash of the parameters and algorithm version. Reused for 24 hours.

- **Location keys:** coordinates are rounded to 0.01°. A box is a 3×3 grid of such points; its daily series is the mean across the nine points for each day.
- **Retries:** requests that return 429 (rate limit) or 500/502/503/504 are retried up to three times with backoff (2 s, 4 s, or the server's `Retry-After`, capped at 10 s). Other failures return HTTP 502 with a readable message.
- **Clamped end date:** the archive lags real time by a few days, so the end date is never later than today minus two days.

4. Data cleaning and handling of gaps

Issue	Handling
Implausible temperature (outside −60 to 60 °C)	Set to NULL and treated as missing.
Negative precipitation	Set to NULL.
Missing days (null values or absent dates)	Never filled or interpolated. A missing day is not hot, not heavy-rain and not dry, so it breaks a run instead of joining two spells.
Years with under 80% daily coverage	Kept in the yearly table, flagged as excluded, and left out of counts and trend statistics. Listed in the data-quality panel.
Box with some missing grid values on a day	The daily mean uses only the points that have a value, so one missing point does not blank the whole day.
Future or partial current year	Rejected for future years; a partial current year is usually excluded by the 80% coverage rule.

5. Hazard definitions

All hazards use the same event detector: a run of consecutive flagged days of at least *min_days* is one event. Each event is attributed to the year in which it starts.

Hazard	Flagged day	Default min. days	Intensity measure
Heatwave	Daily Tmax above the 95th percentile for that calendar day, from the baseline period (default 1981–2010). The percentile pools each baseline year's values within ±7 days of the calendar day. Days with fewer than 30 baseline samples cannot be flagged.	3	Mean °C above the threshold across event days
Heavy rainfall	Daily precipitation above the 95th percentile of baseline wet days (≥ 1 mm). One threshold for the whole year.	1	Mean mm above the threshold
Drought (dry spell)	Daily precipitation below 1 mm.	10	Not defined; spell duration is the measure

> **Why a calendar-day threshold?** A single annual 95th percentile would flag most of July and miss the cool-season extremes. Using the percentile for each time of year compares each day with its own season, which is the standard approach in heatwave studies.

6. Trend analysis

- **Yearly metrics:** event count, event days, longest event, mean intensity, data coverage.
- **Change over the period:** mean event count of the first five valid years compared with the last five. This drives the headline insight, for example "Heatwave events have increased by X% in Muscat since 1990". It is reported as a percentage with its inputs shown next to it.
- **Ordinary least squares (OLS):** slope per year (reported per decade) and R^2.
- **Mann–Kendall test:** two-sided, normal approximation, with a tie correction. Gives τ, p-value and a trend label at the 5% level. Needs at least four valid years.
- **Sen's slope:** median of all pairwise slopes, a robust magnitude estimate reported per decade.
- **5-year rolling mean** of counts, plotted with the bars.

Insights are generated from these statistics and state the significance level, the number of excluded years, the threshold definition, and a caveat that baseline years are included in the analysis.

7. API

Method and path	Purpose
`GET /api/health/`	Liveness check.
`GET /api/geocode/?q=`	City search via Open-Meteo geocoding. Cached 24 hours. Query length 2–100.
`GET /api/analysis/`	Runs the analysis. Parameters: `lat`, `lon`, optional `box_size` (0.05–2.5° half-width), `hazard`, `start_year`, `end_year`, `percentile` (50–99.9), `min_days` (1–30), `baseline_start`, `baseline_end`, `label`. Returns JSON.
`GET /api/analysis/export/`	Same parameters; returns the yearly table as CSV.

Response (abridged)

```
{
  "hazard": "heatwave", "hazard_label": "Heatwave",
  "location": {"label": "Muscat", "lat": 23.61, "lon": 58.54},
  "events_total": 106,
  "percent_change_first_last_5yr": 638.0,
  "trend": {"slope_per_decade": 2.97, "r_squared": 0.34,
            "mann_kendall": {"tau": ..., "p_value": 0.011, "trend": "increasing",
"sen_slope_per_decade": ...}},
  "data_quality": {"coverage_pct": 100.0, "years_excluded": [], "valid_days": 366},
  "plot": {"years": [...], "event_count": [...], "rolling_mean_5yr": [...], "event_days": [...],
"mean_intensity": [...]},
  "yearly": [...], "insights": ["..."], "cached": false
}
```

Input validation and limits

- Latitude and longitude ranges; years from 1940 to the current year; start before end; span up to 80 years.
- Baseline must be at least 10 years, end by the current year, and start before it ends.
- Hazard is restricted to an allow-list. Unknown fields are ignored. All database access goes through the ORM, so no user input is placed into raw SQL.

Rate limiting and caching

- Anonymous default: 60 requests per hour. Analysis endpoint: 20 per hour. Both are configurable by environment variable. Over the limit returns HTTP 429.
- Analysis responses carry `Cache-Control: public, max-age=3600`. Stored results are reused for 24 hours.
- Upstream errors are returned as 502 with a message rather than a stack trace.

8. Database design (MySQL)

Table	Role	Key constraints
`hazards_dailyweather`	Cleaned daily values per location	Unique (location_key, date); bulk upsert on conflict
`hazards_fetchedrange`	Downloaded date ranges per location	Indexed on location and dates
`hazards_analysisresult`	JSON result document per parameter set	Unique cache_key; JSON column for params and result

The case brief mentions a NoSQL store. The implementation uses MySQL, as requested for this build. The `AnalysisResult` table stores documents as JSON, so the read path (“read from the database and return JSON”) is satisfied.

9. Frontend

- **Region picker:** three modes — city search (with debounce), latitude/longitude, and lat-lon box (centre and half-width, 0.25–2°). A Leaflet map sets the point or box centre on click.
- **Hazard and period:** hazard selector (heatwave default), from/to year inputs. An advanced panel exposes percentile, minimum days and baseline years when relevant.
- **Client-side validation** before any request, with messages listed under the form.
- **Results:** KPI tiles (events, change over the period, trend per decade, Mann–Kendall p-value); insights list; bar and rolling-mean chart of counts; duration and intensity chart; data-quality panel.
- **Exports:** Download CSV (from the API) and Print / save as PDF (print stylesheet hides the form).
- **Error handling:** readable messages for unreachable backend, rate limit and upstream failure. Previous results are cleared when a new analysis starts.
- Dark mode follows the operating system preference.

10. Testing and verification

Area	What is checked
Heatwave rules	Calendar-day threshold matches an independent percentile calculation; runs of 2 vs 3 days; a missing day breaks a run; a heavy-rain or drought spell needs the minimum length.
Threshold regression	The temperature threshold uses only Tmax, not Tmin or precipitation. An earlier version pooled the columns and inflated counts; this test prevents a repeat.
Ingestion	Implausible temperatures and negative rain become NULL; a cached range is not fetched again.
Trends	Mann–Kendall detects a clear trend and reports no trend for a flat series; Sen's slope and OLS agree on linear data; percentage change handles a zero baseline.
API	Validation rejects inverted ranges, future years, out-of-range coordinates, unknown hazards and spans over 80 years; the second identical call is served from the database; CSV export returns the expected rows; the 3rd call beyond a limit of 2 returns 429.
Live backend	Heatwave, heavy-rain and drought analyses for Muscat (1990–2020) return HTTP 200 with consistent output.
Browser (Microsoft Edge, Playwright)	The heatwave flow renders KPIs, insights and charts. The heavy-rain box flow renders results from the nine-point grid.
Build	Production frontend build completes without errors. Backend test suite: 21 tests, all passing.

Issues found and fixed during development

- **Pooled percentile:** the first threshold function pooled all daily columns into one percentile, which inflated heatwave counts. Fixed and covered by a regression test.
- **MySQL upsert:** the bulk insert used an option that MySQL does not support; replaced with the MySQL-compatible form.
- **Mann–Kendall with few years:** the insight text formatted a missing p-value; now states that there are too few years for a test.

- **Upstream rate limit:** the first version failed immediately on HTTP 429; it now retries with backoff.

11. Assumptions and limitations

- **Reanalysis data:** ERA5 is a model-based product at about 0.25° resolution. It can smooth local extremes compared with station records.
- **Baseline overlap:** baseline years are part of the analysed series, so their exceedance rate is close to, but not exactly, 5% by construction. The insights state this.
- **Percentage change is volatile:** it compares two five-year averages of small counts. Large percentages can come from small absolute changes, so the regression and Mann–Kendall results are shown beside it.
- **Box as a sample:** a 3×3 grid describes the area coarsely. It is an approximation, not a full spatial average.
- **Leap years:** the calendar lookup uses day-of-year, so 29 February shares a threshold slot with 1 March. The effect on the percentile is negligible.
- **External quota:** the Open-Meteo free tier applies rate limits. Uncached regions can be temporarily unavailable; the retry logic reduces but does not remove this.
- **Intensity for drought** is not defined; only duration is reported.
- **Local deployment:** the rate limiter and the response cache use in-process memory, so limits are per process. Multiple workers would need Redis.

12. Possible next steps

- Finer spatial sampling for boxes, or gridded ERA5 files instead of nine point requests.
- Show event locations on the map, not only the selected region.
- Scheduled refresh of recent data instead of fetch-on-demand.
- Redis for shared cache and rate limits; bootstrap confidence intervals for the percentage change.
- Frontend component tests and a CI pipeline.

13. Running the project

```
# Database: create an empty MySQL/MariaDB database (utf8mb4).

# Backend
cd backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env        # set DB_* and DJANGO_SECRET_KEY
python manage.py migrate
python manage.py test hazards
python manage.py runserver 8000

# Frontend
cd frontend
npm install
npm run dev                   # http://localhost:5173, proxies /api to port 8000
```

Configuration is read from environment variables: database connection (`DB_*`), Django secret and debug flags, allowed hosts and CORS origins, and rate limits. No credentials are stored in the code.

14. AI tools used

Claude Code (Anthropic) was used to scaffold the project, write the hazard logic, tests and UI from a written specification, and to run the tests and the browser checks. The author reviewed the generated code, ran the verification steps above, corrected the issues listed in section 10, and made the definitional choices in sections 5 and 11.